The *Reactive Manifesto* (www.reactivemanifesto.org) describes a reactive system as responsive, resilient, elastic, and message-driven. Its mission to build loosely coupled, scalable, resilient, and cost-effective applications is fully compatible with our definition of cloud native. The new part is achieving that goal by using an asynchronous and non-blocking communication paradigm based on message-passing.

Before diving into building reactive applications in Spring, we'll explore the basics of reactive programming, why it matters for cloud native applications, and how it differs from imperative programming. I'll introduce the event loop model, which overcomes the thread-per-request model's shortcomings. Then you'll learn the essential concepts of the Reactive Streams specification implemented by Project Reactor and the Spring reactive stack.

8.1.1 From thread-per-request to event loop

As you saw in chapter 3, non-reactive applications allocate a thread per request. Until a response is returned, the thread will not be used for anything. That is the *thread-per-request* model. When the request handling involves intensive operations like I/O, the thread will block until those operations are completed. For example, if a database read is required, the thread will wait until data is returned from the database. During the waiting time, the resources allocated to the handling thread are not used efficiently. If you want to support more concurrent users, you'll have to ensure you have enough threads and resources available. In the end, this paradigm sets constraints on the application's scalability and doesn't use computational resources in the most efficient way possible. Figure 8.1 shows how it works.